

WARING DECOMPOSITION OF DERIVATIVES AND ITS APPLICATIONS

AUTHOR ONE* AND AUTHOR TWO†

Abstract. Computing high-order derivatives in neural network residuals is often computationally and memory intensive, primarily due to the huge computational graphs caused by automatic differentiation. In this work, we decompose the derivative into a weighted sum of directional Taylor coefficients. We prove that the minimal number of directional evaluations required to recover a target partial derivative equals the Waring rank of its corresponding monomial, and we attain this minimum with a roots-of-unity formula. Experiments show that our method significantly outperforms nested automatic differentiation in both speed and memory efficiency.

Key words. high-order derivatives, Taylor-mode automatic differentiation, Waring decomposition, apolarity

AMS subject classifications. 65D25, 14N07

1. Introduction. Physics-informed neural networks approximate solutions of differential equations by minimizing loss functions constructed from the corresponding PDE residuals. This requires derivatives of the network with respect to its inputs. For high-order equations, the repeated evaluation of these derivatives can dominate both the runtime and memory cost of residual evaluation.

Automatic differentiation is the standard tool for evaluating these derivatives [1, 9, 22]. Several exact approaches reduce its cost by exploiting the structure of the differential operator. For example, forward Laplacian methods propagate the Laplacian through the network without constructing the full Hessian [20]. Such methods can be efficient, but their differentiation rules are designed for specific structured operators and do not readily extend to general high-order partial derivatives. Taylor-mode automatic differentiation provides a more general alternative by propagating a truncated univariate Taylor series through the network [2, 9]. Existing general constructions recover multivariate high-order derivatives from multiple Taylor expansions, often by reconstructing the full derivative tensor [8]. The number of tensor components grows rapidly with both the input dimension and the derivative order, resulting in substantial computational and GPU-memory costs [26].

Randomization offers another route to reducing derivative costs. Stochastic dimension gradient descent uses coordinate sampling to evaluate only a subset of derivative terms during each optimization step [13]. Hutchinson-type estimators use random projections to approximate trace-type differential operators [11, 14, 23]. However, both approaches still rely on automatic differentiation to compute the sampled derivatives or projected vector products. A separate line of work avoids automatic differentiation entirely when computing derivatives. Finite differences approximate derivatives from nearby network evaluations [4, 6], whereas randomized smoothing uses Stein’s identity to express derivatives of a smoothed network as expectations of perturbed function values [10, 29]. STDE takes a different route by combining randomization with high-order automatic differentiation: it constructs randomized input tangents so that general linear differential operators can be estimated through univariate Taylor-mode propagation [26]. STDE++ also discusses the exact recovery of mixed partial

*Institution One (author1@example.com).

†Institution Two (author2@example.com).

derivatives from linear combinations of directional Taylor coefficients. It further develops STDE by providing polynomial-time Taylor-jet constructions, improving on earlier general schemes whose cost grows exponentially with the derivative order [27]. Although these methods reduce computational costs, their performance depends on the choice of method-specific parameters. Finite differences require a step size that balances truncation and round-off errors, randomized smoothing depends on both the smoothing bandwidth and the Monte Carlo sample size, and the randomized estimators require a sampling distribution and budget. For the stochastic methods, these choices directly affect estimator variance: insufficient or poorly matched sampling can yield high-variance residual estimates, while a small smoothing bandwidth can sharply amplify variance for high-order derivatives and may destabilize PINN training [10, 12, 27].

In this paper, we propose an exact method for recovering a partial derivative as a linear combination of directional Taylor coefficients with the minimum possible number of directions. Our main contributions are:

- We establish an equivalence between Waring decompositions of a monomial and decompositions of the corresponding partial derivative into directional Taylor coefficients. The equivalence shows that the minimum number of directions equals the Waring rank of the monomial.
- We derive an explicit roots-of-unity formula attaining the minimum number of directions. A batched Taylor-mode implementation evaluates all directional Taylor coefficients independently and in parallel.
- We compare the proposed method with nested automatic differentiation on manufactured PINN benchmarks. The experiments show reductions in runtime and peak memory, with larger reductions at higher derivative orders.

The remainder of the paper is organized as follows. Section 2 introduces the method of PINNs, the multi-index notation, and the directional Taylor coefficient. Section 3 proves the equivalence with monomial Waring rank and derives the rank-optimal formula. Section 4 describes its realization with batched Taylor jets. Section 5 compares the implementation with backpropagation applied p times on manufactured residuals. Section 6 concludes. The roots-of-unity formula, the schedules used in the experiments, and the argument for the rank bound are collected in Appendix A.

2. Preliminaries.

2.1. Method of PINNs. Let $\Omega \subset \mathbb{R}^d$ be a computational domain with boundary $\Gamma := \partial\Omega$. We consider the boundary-value problem

$$\mathcal{P}u_\star = f \quad \text{in } \Omega, \quad \mathcal{B}u_\star = g \quad \text{on } \Gamma, \quad (1)$$

where $\mathcal{P}$ is a differential operator and $\mathcal{B}$ denotes a boundary operator or a collection of boundary operators. In the PINN framework, the solution $u_\star$ is approximated by a neural network u_θ [17, 18, 24, 28]. Given residual points $(\mathbf{x}_{r,i})_{i=1}^{n_r} \subset \Omega$ and boundary points $(\mathbf{x}_{b,i})_{i=1}^{n_b} \subset \Gamma$, together with positive weights λ_r and λ_b , the network parameters are obtained by minimizing

$$\mathcal{L}(\theta) := \frac{\lambda_r}{n_r} \sum_{i=1}^{n_r} |\mathcal{P}u_\theta(\mathbf{x}_{r,i}) - f(\mathbf{x}_{r,i})|^2 + \frac{\lambda_b}{n_b} \sum_{i=1}^{n_b} \|\mathcal{B}u_\theta(\mathbf{x}_{b,i}) - g(\mathbf{x}_{b,i})\|^2. \quad (2)$$

Evaluating the residual term in (2) requires derivatives of u_θ . For high-order equations and high-dimensional problems, nested first-order automatic differentiation can

dominate the training cost because it retains large intermediate computational graphs and performs repeated differentiation passes through the network.

2.2. Multi-index notation and the target derivative. We first fix the notation used below.

Let $\mathbb{N}_0 := \{0, 1, 2, \dots\}$, and let $\mathbb{C}^d$ denote the d -dimensional complex space. For a multi-index $\nu \in \mathbb{N}_0^d$ and a vector $\mathbf{z} \in \mathbb{C}^d$, define $\partial_i := \partial/\partial x_i$ and write

$$|\nu| := \sum_{i=1}^d \nu_i, \quad \nu! := \prod_{i=1}^d \nu_i!, \quad \mathbf{z}^\nu := \prod_{i=1}^d z_i^{\nu_i}, \quad \partial^\nu := \partial_1^{\nu_1} \cdots \partial_d^{\nu_d}. \quad (3)$$

The integer $p := |\nu|$ is the order of the derivative.

The proposed method evaluates one partial derivative at a time. Fix a point $\mathbf{x} \in \Omega$ and a multi-index ν of order $p \geq 1$. For the derivative analysis, let u denote the scalar function being differentiated; in the PINN setting, $u = u_\theta$. The target derivative is

$$\partial^\nu u(\mathbf{x}) := \partial^\nu u(\mathbf{x}). \quad (4)$$

A PINN residual involving several derivative terms is assembled after each derivative has been evaluated separately.

We regard u as a scalar-valued function from $\mathbb{C}^d$ to $\mathbb{C}$ and assume that it is holomorphic in a complex neighborhood of $\mathbf{x}$. Feed-forward networks composed of affine maps and entire activations, such as $\sinh(z) := (e^z - e^{-z})/2$, satisfy this assumption. The formulation covers both networks whose restriction to $\mathbb{R}^d$ is real-valued and networks used for complex-valued solutions.

2.3. Directional Taylor coefficients. Let $D^p u(\mathbf{x})$ denote the symmetric p -linear derivative of u at $\mathbf{x}$. For a direction $\mathbf{v} \in \mathbb{C}^d$, the directional Taylor coefficient of order p at $\mathbf{x}$ along $\mathbf{v}$ is

$$T_p(\mathbf{x}; \mathbf{v}) := \frac{1}{p!} D^p u(\mathbf{x})[\mathbf{v}, \dots, \mathbf{v}] = \frac{1}{p!} \frac{d^p}{d\tau^p} u(\mathbf{x} + \tau \mathbf{v}) \Big|_{\tau=0}, \quad (5)$$

where $D^p u(\mathbf{x})$ is extended complex-multilinearly. The second equality identifies $T_p(\mathbf{x}; \mathbf{v})$ with the coefficient of order p in the Taylor expansion of the holomorphic path $\tau \mapsto u(\mathbf{x} + \tau \mathbf{v})$.

Taylor-mode automatic differentiation evaluates the Taylor coefficients up to order p in one forward jet propagation [2, 9]. Along the path $\tau \mapsto u(\mathbf{x} + \tau \mathbf{v})$, each intermediate quantity y is represented by the truncated coefficient tuple $(y_0, \dots, y_p)$, where

$$y_k := \frac{1}{k!} \frac{d^k y}{d\tau^k} \Big|_{\tau=0}. \quad (6)$$

The coefficient of order p at the network output is $T_p(\mathbf{x}; \mathbf{v})$. Taylor-mode propagation therefore computes the required directional coefficient without constructing nested input-differentiation graphs.

3. Exact directional representations and Waring rank. Fix $\mathbf{x}$ and ν as in Section 2.2, and let $p = |\nu|$. After permuting and relabeling the active coordinates, write the positive entries of ν as $a = (a_0, \dots, a_n)$, ordered so that

$$1 \leq a_0 \leq \dots \leq a_n, \quad p = \sum_{j=0}^n a_j, \quad \nu! = \prod_{j=0}^n a_j!. \quad (7)$$

Denote the corresponding relabeled standard basis vectors by $e_0, \dots, e_n$. Coordinates outside this set do not contribute to $\partial^\nu u(\mathbf{x})$, and all directions below are therefore supported on the active coordinates. Multi-indices in this section range over these $n + 1$ coordinates, and $\mathbf{z}^a$ denotes the monomial $z_0^{a_0} \cdots z_n^{a_n}$. We first identify exact directional representations with Waring decompositions. The monomial rank formula then gives the minimum number of directions, and a roots-of-unity construction realizes the minimum.

3.1. Directional representations and Waring decompositions. For a direction $\mathbf{v} = \sum_{j=0}^n v_j e_j$, the multinomial theorem and (5) give

$$T_p(\mathbf{x}; \mathbf{v}) = \sum_{|\beta|=p} \frac{\mathbf{v}^\beta}{\beta!} \partial^\beta u(\mathbf{x}). \quad (8)$$

Thus each directional Taylor coefficient is a linear combination of the order- p partial derivatives. An R -term exact directional representation of $\partial^\nu u(\mathbf{x})$ is an identity

$$\partial^\nu u(\mathbf{x}) = \sum_{r=1}^R c_r T_p(\mathbf{x}; \mathbf{v}_r) \quad (9)$$

that holds for every u holomorphic on a complex neighborhood of $\mathbf{x}$, where $c_r \in \mathbb{C}$ and each $\mathbf{v}_r$ is supported on the active coordinates.

PROPOSITION 3.1 (Directional–Waring correspondence). *The coefficients and directions in (9) form an exact directional representation if and only if*

$$p! \mathbf{z}^a = \sum_{r=1}^R c_r (\mathbf{v}_r \cdot \mathbf{z})^p \quad \text{in } \mathbb{C}[z_0, \dots, z_n]. \quad (10)$$

Proof. Substituting (8) into (9), the coefficient multiplying $\partial^\beta u(\mathbf{x})$ on the right-hand side is $\beta!^{-1} \sum_{r=1}^R c_r \mathbf{v}_r^\beta$. Testing the identity on degree- p polynomials in the shifted coordinates shows that these coefficients must agree separately. Hence (9) holds for every such u if and only if

$$\sum_{r=1}^R c_r \mathbf{v}_r^\beta = \nu! \delta_{a\beta}, \quad |\beta| = p, \quad (11)$$

where $\delta_{a\beta} = 1$ for $\beta = a$ and is zero otherwise. The multinomial theorem also gives

$$\sum_{r=1}^R c_r (\mathbf{v}_r \cdot \mathbf{z})^p = \sum_{|\beta|=p} \binom{p}{\beta} \left(\sum_{r=1}^R c_r \mathbf{v}_r^\beta \right) \mathbf{z}^\beta.$$

The moment conditions (11) leave only the term $\beta = a$, whose coefficient is $\binom{p}{a} \nu! = p!$. Conversely, comparison of the coefficients of $\mathbf{z}^\beta$ in (10) recovers (11). $\square$

The restriction to active directions does not change the minimum. An exact representation using directions in the ambient space gives a Waring identity for the target monomial in all d variables. Setting the inactive polynomial variables to zero projects the identity onto the active coordinates without increasing the number of summands.

Equation (10) is a Waring decomposition of the scalar multiple $p!z^a$. We recall the corresponding rank.

DEFINITION 3.2 (Waring rank). *Let $F \in \mathbb{C}[z_0, \dots, z_n]$ be homogeneous of degree p . The complex Waring rank $R_{\mathbb{C}}(F)$ is the least integer R for which there exist coefficients $c_r \in \mathbb{C}$ and linear forms L_r satisfying*

$$F = \sum_{r=1}^R c_r L_r^p.$$

Multiplication by a nonzero scalar does not change Waring rank. Therefore, Proposition 3.1 identifies the minimum number of directions in (9) with $R_{\mathbb{C}}(z^a)$. The monomial rank theorem supplies the minimum, while the construction below specifies directions and weights that attain it. For $m \geq 1$, let $U_m := \{\zeta \in \mathbb{C} : \zeta^m = 1\}$.

THEOREM 3.3 (Rank-optimal directional representation). *Let $1 \leq a_0 \leq \dots \leq a_n$ be the active exponents of ν as in (7). The minimum number of terms in an exact directional representation of $\partial^\nu u(\mathbf{x})$ is*

$$R_{\mathbb{C}}(z^a) = \prod_{j=1}^n (a_j + 1). \quad (12)$$

For $\zeta = (\zeta_1, \dots, \zeta_n) \in U_{a_1+1} \times \dots \times U_{a_n+1}$, define

$$\mathbf{v}_\zeta := e_0 + \sum_{j=1}^n \zeta_j e_j, \quad c_\zeta := \frac{\nu!}{\prod_{j=1}^n (a_j + 1)} \prod_{j=1}^n \zeta_j. \quad (13)$$

Then

$$\partial^\nu u(\mathbf{x}) = \sum_{\zeta \in U_{a_1+1} \times \dots \times U_{a_n+1}} c_\zeta T_p(\mathbf{x}; \mathbf{v}_\zeta). \quad (14)$$

The representation (14) attains the minimum in (12), even when directions in the ambient space $\mathbb{C}^d$ are allowed.

Proof. Proposition 3.1, invariance of Waring rank under nonzero scaling, and the preceding projection argument identify the minimum over ambient directions with $R_{\mathbb{C}}(z^a)$. The monomial rank theorem of Carlini, Catalisano, and Geramita [3] gives (12). Appendix A.3 records the apolar interpretation.

It remains to verify that the roots-of-unity construction attains the minimum. For $|\beta| = p$, summing over $\zeta \in U_{a_1+1} \times \dots \times U_{a_n+1}$ gives

$$\sum_{\zeta} c_\zeta \mathbf{v}_\zeta^\beta = \frac{\nu!}{\prod_{j=1}^n (a_j + 1)} \prod_{j=1}^n \left(\sum_{\zeta_j \in U_{a_j+1}} \zeta_j^{\beta_j+1} \right). \quad (15)$$

The roots-of-unity filter in Lemma A.1 shows that the j th factor is zero unless

$$\beta_j \equiv a_j \pmod{a_j + 1}, \quad j = 1, \dots, n.$$

Nonnegativity then implies that each β_j is either a_j or at least $2a_j + 1$. If the second alternative holds for an index $j \geq 1$, all filtered exponents satisfy $\beta_k \geq a_k$ for $k = 1, \dots, n$. The excess at the index j is at least $a_j + 1$. Therefore,

$$\beta_0 = p - \sum_{k=1}^n \beta_k \leq a_0 - (a_j + 1) < 0,$$

where the last inequality follows from $a_0 \leq a_j$. This contradicts $\beta_0 \geq 0$. Consequently, the product in (15) can be nonzero only for $\beta = a$. For $\beta = a$, every factor equals $a_j + 1$, and (15) equals $\nu!$. The moment conditions (11) therefore hold, which proves (14). When $n = 0$, the formula reduces to $\partial_0^p u(\mathbf{x}) = p! T_p(\mathbf{x}; e_0)$. $\square$

3.2. Examples and consequences. In the examples below, we return to the original coordinate labels. For the mixed second derivative, $a = (1, 1)$ and (12) gives rank two. Formula (14) becomes

$$\partial_1 \partial_2 u(\mathbf{x}) = \frac{1}{2} T_p(\mathbf{x}; e_1 + e_2) - \frac{1}{2} T_p(\mathbf{x}; e_1 - e_2). \quad (16)$$

Since $R_{\mathbb{C}}(z_0 z_1) = 2$, no exact representation of the mixed second derivative by a single directional Taylor coefficient exists.

For a genuinely complex example, consider $a = (2, 2)$ and let $\omega = \exp(2\pi i/3)$. Then

$$\partial_1^2 \partial_2^2 u(\mathbf{x}) = \frac{4}{3} \sum_{k=0}^2 \omega^k T_p(\mathbf{x}; e_1 + \omega^k e_2), \quad (17)$$

and three directions are minimal over $\mathbb{C}$.

The rank formula also identifies the limits of the reduction. A pure power $\partial_i^p u$ has rank one, whereas a square-free derivative with $a = (1, \dots, 1)$ has rank 2^{p-1} . Between these cases, repeated indices can reduce the rank below the square-free value. Table 3.1 lists representative values, and Appendix A.2 records the formulas used in the experiments.

The minimum in Theorem 3.3 concerns one fixed monomial derivative and exact linear representations by order- p directional Taylor coefficients. A differential operator containing several monomials is evaluated term by term. By contrast, the stochastic Taylor derivative estimator [26] uses a sampling budget to estimate arbitrary contractions of the order- p derivative tensor. The stochastic estimation problem is distinct from the deterministic minimum in (12).

TABLE 3.1

The minimum number of directional evaluations $R_{\mathbb{C}}(\mathbf{z}^a)$ of (12), for selected active-exponent patterns up to $p = 6$. Repeated indices reduce the count relative to the square-free pattern at the same order.

pattern	example multi-index	$R_{\mathbb{C}}$
(p)	$\partial_1^p u$	1
$(2, 1)$	u_{112}	3
$(1, 1, 1)$	u_{123}	4
$(3, 1)$	u_{1112}	4
$(2, 2)$	u_{1122}	3
$(2, 1, 1)$	u_{1123}	6
$(1, 1, 1, 1)$	u_{1234}	8
$(4, 2)$	u_{111122}	5
$(2, 2, 2)$	u_{112233}	9
(1^6)	u_{123456}	32

4. Computational realization.

4.1. Batched Taylor-jet evaluation. Algorithm 1 assembles the pieces. The schedule (13) is built once, its directions are stacked into a single batch, and one jet pass through the function returns the order- p coefficient for every direction at once; the reported value is then the coefficient-weighted sum. The schedule tensors depend on ν , the device, and the scalar type, but not on θ . The reference implementation builds them at call entry and includes that work in the measured throughput of Section 5, while a caller that reuses one multi-index may cache them. For a real-valued function the conjugate pairing of the roots of unity makes the returned value real in exact arithmetic; the implementation keeps the complex tensor and takes its real part only when the target PDE is real-valued.

The roots-of-unity directions are generally complex, so the network is evaluated on complex Taylor jets. Entire activations such as $\sinh$ and $\cosh$ are holomorphic on all of $\mathbb{C}$. By contrast, $\tanh$ is meromorphic, with poles at $i\pi/2 + ik\pi$ for $k \in \mathbb{Z}$, and is admissible only when the relevant complex paths avoid those poles. The experiments use $\sinh$ at every derivative order.

Algorithm 1 Single-monomial $\partial^\nu u(\mathbf{x})$ from a rank-optimal schedule and one batched Taylor jet.

Require: scalar network u built from supported analytic primitives; point $\mathbf{x} \in \mathbb{R}^d$; multi-index ν of order p

Ensure: the value $\partial^\nu u(\mathbf{x})$

- 1: Relabel the active basis vectors as $e_0, \dots, e_n$ with exponents $1 \leq a_0 \leq \dots \leq a_n$.
 - 2: Set $\mathcal{I} := U_{a_1+1} \times \dots \times U_{a_n+1}$.
 - 3: Initialize $\mathcal{S} \leftarrow \emptyset$.
 - 4: **for** $\zeta = (\zeta_1, \dots, \zeta_n) \in \mathcal{I}$ **do**
 - 5: Set $\mathbf{v}_\zeta \leftarrow e_0 + \sum_{j=1}^n \zeta_j e_j$ and $c_\zeta \leftarrow \nu! (\prod_{j=1}^n (a_j + 1))^{-1} \prod_{j=1}^n \zeta_j$.
 - 6: Update $\mathcal{S} \leftarrow \mathcal{S} \cup \{(c_\zeta, \mathbf{v}_\zeta)\}$.
 - 7: **end for**
 - 8: Stack the directions of $\mathcal{S}$ into $V_\mathcal{S} \in \mathbb{C}^{|\mathcal{S}| \times d}$.
 - 9: Form the batched input jet $\mathbf{J}_0 \leftarrow (\mathbf{x} \text{ replicated}, V_\mathcal{S}, 0, \dots, 0)$.
 - 10: Propagate $\mathbf{J}_0$ through u by Taylor-mode automatic differentiation to obtain $\mathbf{J}_L$.
 - 11: Set $t \leftarrow [\mathbf{J}_L]_p$, one order- p coefficient per direction.
 - 12: **return** $\sum_{r=1}^{|\mathcal{S}|} c_r t_r$.
-

4.2. Parameter gradients. Algorithm 1 must be differentiable in θ , since its output enters a residual assembled from monomial derivatives. Differentiating the activation jets by reverse-mode automatic differentiation through an unrolled recurrence would rebuild inside the backward pass the nesting that the jet was introduced to avoid. We therefore register each activation jet as a single primitive with the closed-form adjoint

$$g_x^{(m)} = \sum_{j=0}^{p-m} \overline{q_j} g_y^{(m+j)}, \quad q(\tau) := \phi'(x(\tau)), \quad (18)$$

in which q_j is the j th Taylor coefficient of q and $g^{(m)}$ denotes the cotangent of the order- m coefficient. Equation (18) follows from the chain rule applied to the jet of ϕ and uses the convention of a real-valued loss for complex parameters, so that the conjugation is the Hermitian adjoint of multiplication by q_j . The backward pass thus

costs one more triangular convolution per activation and never differentiates through the recurrence itself.

4.3. Computational cost. The cost of Algorithm 1 separates into a per-direction and a per-schedule factor. For one direction, the jet stores $p + 1$ coefficient tensors per activation, an affine layer applies W to each of them, and the activation recurrences require $\sum_{k=1}^p O(k) = O(p^2)$ elementwise products per scalar. The total work and the directional storage then scale linearly in the schedule length $R_{\mathbb{C}}(\mathbf{z}^a)$, which by Theorem 3.3 is as small as an exact directional formula can be. The rank reduction is an exact statement about that factor and not about wall-clock time, which also depends on complex arithmetic, kernel launch overhead, memory traffic, and whether parameter gradients are required; for small widths or small batches these terms can dominate. Section 5 therefore reports measured throughput alongside accuracy.

The schedule coefficients are known in closed form. The remaining error is floating-point round-off from the Taylor recurrences, from complex arithmetic, and from the final summation over directions. Section 5.1 reports double-precision agreement through order six.

5. Numerical experiments. The experiments evaluate monomials that appear in the PINN residual (2) for the boundary-value problem (1). The benchmark source terms and boundary data are manufactured from prescribed exact solutions [25]. Expanding $\mathcal{P}$ into monomials turns one residual evaluation in (2) into several derivatives $\partial^\alpha u_\theta$ of the same function on the same batch. The operator Δ^m expands into $\binom{d+m-1}{m}$ distinct monomials: the pure terms $\partial_{x_i}^{2m}$ and, for $m \geq 2$, mixed terms such as $\partial_{x_i}^{2k} \partial_{x_j}^{2m-2k}$.

This section checks two properties of the schedule: agreement with nested automatic differentiation in double precision, and the time and memory of one residual evaluation, including the reverse-mode pass. The first comparison uses no PDE. The second repeats the same evaluation inside an otherwise identical PINN training loop and records the time and memory of one step. Every quantity below is a property of the derivative method on a fixed function, residual, and optimizer.

5.1. Verification of the derivative backend. All checks use double precision and are executable tests in the repository. The roots-of-unity coefficient filter of Lemma A.1 is evaluated against every degree-four monomial in five variables for the active-exponent pattern $(2, 1, 1)$, with a maximum admissible absolute coefficient error of 10^{-12} . The length of the schedule that the implementation builds is checked against the closed-form rank (12) for every pattern of Table 3.1.

The Waring jet of Algorithm 1 is then compared with nested automatic differentiation on real-parameter networks using the tanh, logistic, sin, and sinh activations, for the patterns $(2, 2)$, $(1, 1, 1)$, and (3) . Values must agree to relative tolerance 2×10^{-10} and absolute tolerance 2×10^{-11} , and the imaginary remainder of a real target must not exceed 2×10^{-11} . Parameter gradients of a squared derivative loss are compared at tolerances 2×10^{-9} relative and 2×10^{-11} absolute, on a real tanh network and on a complex sinh network; additional analytic checks cover the orders 1, 2, 3, 4, and 6 for a scalar complex sinh network. Every check passes at the tolerance stated for it. These tests establish that the implementation computes what Theorem 3.3 predicts.

5.2. A controlled comparison of derivative backends. The remaining study isolates the derivative backend. Protocol `jsc_v3` runs two lines that share the network, the loss, the optimizer, the residual points, the evaluation rule, the seeds,

and the wall-clock budget, and differ in one respect: how each monomial derivative of the residual is evaluated. The first line uses Algorithm 1, that is, the rank-optimal complex schedule evaluated by one batched Taylor jet, and is labeled *jet* below. The second evaluates the same derivative of the same network by nested reverse-mode automatic differentiation, with the graph retained after every coordinate derivative, and is labeled *nested*. Both lines are the same complex-parameter sinh network with four hidden layers of width $H = 128$, hence the same 100,098 trainable real degrees of freedom, the same first-layer scale, and the same `complex128` arithmetic. Both use Adam with initial learning rate 10^{-3} and cosine decay, 4096 fixed interior points, 512 fixed boundary points, paired point sets for each seed, the same per-setting boundary weights, and a common held-out set of 8192 points. Each line is run for three seeds with a budget of 1000 seconds per run.

A *setting* is one problem together with one value of its sweep parameter, and the protocol covers nine: the two-dimensional polyharmonic problem at orders $2m \in \{2, 4, 6\}$, the radial chirp at $a \in \{1, 2, 3\}$, and the time-harmonic Maxwell problem at $a \in \{2, 4, 6\}$, all defined as in Section 5.3. All 54 runs were produced by one source commit on one NVIDIA H20 GPU with PyTorch 2.5.1 and CUDA 12.1, so unlike a comparison assembled from separate campaigns, timings here are comparable across settings as well as within one. Cost is the measured time per optimizer step together with the peak memory allocated by a run, and accuracy is the held-out relative L^2 error

$$\mathcal{E}_{L^2} := \frac{\|u_\theta - u_\star\|_{L^2(\Omega)}}{\|u_\star\|_{L^2(\Omega)}}, \quad (19)$$

evaluated on that held-out set against the exact solution $u_\star$. A setting enters the tables only after automated checks confirm all six method-seed rows, finite metrics, nonempty monotone histories, and the protocol metadata.

5.3. Benchmark problems. The three families place different demands on the derivative. For the polyharmonic problem [5, 7] on $(-1, 1)^2$ we solve

$$\Delta^m u = (-2\pi^2)^m u_\star \quad \text{in } (-1, 1)^2, \quad u_\star(\mathbf{x}) = \sin(\pi x_1) \sin(\pi x_2), \quad (20)$$

at the orders $2m \in \{2, 4, 6\}$, with the Navier boundary targets $u = u_\star$ and $\Delta^j u = (-2\pi^2)^j u_\star$ for $j = 1, \dots, m-1$. Those traces vanish on $\partial(-1, 1)^2$ because $u_\star$ does. This family is the reason the derivative order appears as a variable: expanding Δ^m in two dimensions produces the patterns $(2m)$ and, for $m \geq 2$, the mixed patterns whose schedules Table 3.1 lists. At $2m = 4$ the mixed term is $\partial_1^2 \partial_2^2$, which (13) evaluates with three directions; at $2m = 6$ the mixed terms are $\partial_1^4 \partial_2^2$ and $\partial_1^2 \partial_2^4$, which take five each.

The other two families sit at order two and sweep a frequency instead. On $(-1, 1)^2$ the radial chirp [16] is

$$-\Delta u + u = f, \quad u_\star(x, y) = \sin\left(\frac{a\pi}{2}(x^2 + y^2)\right), \quad (21)$$

with Dirichlet data from $u_\star$, $a \in \{1, 2, 3\}$, and the manufactured source $f = (a\pi)^2 r^2 \sin \phi - 2a\pi \cos \phi + \sin \phi$, where $\phi := a\pi(x^2 + y^2)/2$ and $r^2 := x^2 + y^2$. Its local radial wavenumber $|\nabla \phi| = a\pi r$ varies over the domain, so the target is not a separable Fourier mode. For a two-dimensional transverse-magnetic mode, Maxwell's

equations [21] reduce to a scalar complex Helmholtz equation for the out-of-plane field E ,

$$\Delta E + \kappa^2 E = f, \quad \kappa^2 = (a\pi)^2(1 + i\beta), \quad E_* = \exp(ia\pi(x + y)), \quad (22)$$

with $\beta = 0.2$, exact Dirichlet data, $a \in \{2, 4, 6\}$, and $f = [-2(a\pi)^2 + \kappa^2]E_*$. The Maxwell family has a complex-valued solution. The measurements already propagate complex tensors through a complex-parameter sinh network, so a complex residual uses the same jet as the real ones. Since both families are of order two, every monomial term of their residuals is a pure power, which the schedule evaluates with a single real direction. They therefore isolate the cost of the Taylor jet on one direction.

TABLE 5.1

Cost of one optimizer step under the jsc_v3 protocol, which changes only the derivative backend. Times are the three-seed mean of milliseconds per step and memory is the peak allocation of a run; the ratio columns give nested automatic differentiation divided by the Taylor jet, so larger is better for the jet.

setting	p	ms per step			peak memory (MiB)		
		jet	nested	ratio	jet	nested	ratio
Polyharmonic, $2m = 2$	2	62.3	91.1	1.46	509	789	1.55
Polyharmonic, $2m = 4$	4	251.6	565.9	2.25	1759	5400	3.07
Polyharmonic, $2m = 6$	6	854.2	3496.1	4.09	5728	36976	6.46
Radial chirp, $a = 1$	2	72.8	101.7	1.40	548	852	1.55
Radial chirp, $a = 2$	2	73.4	101.9	1.39	548	852	1.55
Radial chirp, $a = 3$	2	73.4	102.1	1.39	548	852	1.55
Maxwell, $a = 2$	2	73.3	102.1	1.39	573	853	1.49
Maxwell, $a = 4$	2	73.2	101.9	1.39	573	853	1.49
Maxwell, $a = 6$	2	72.9	101.8	1.40	573	853	1.49

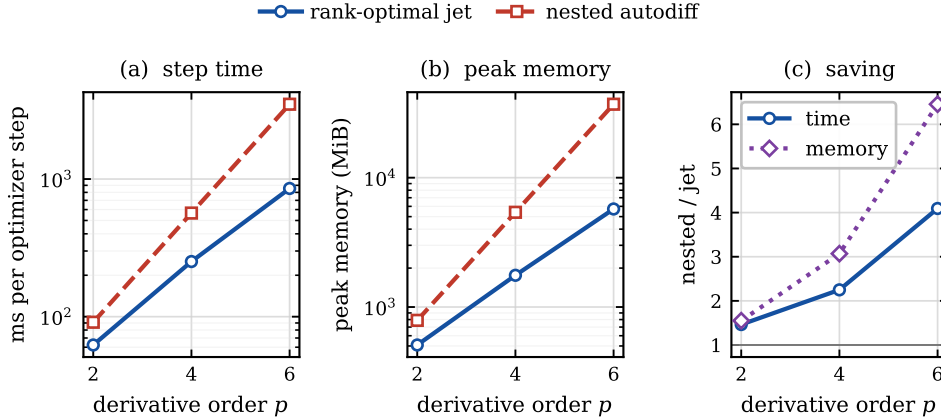


FIG. 5.1. *Cost of one optimizer step against the derivative order, on the polyharmonic family, for the two backends of Section 5.2. Panels (a) and (b) show the three-seed mean step time and the peak memory of a run; panel (c) shows the ratio of nested automatic differentiation to the Taylor jet for both quantities.*

5.4. Cost of one derivative evaluation. Table 5.1 and Figure 5.1 report the cost of one optimizer step. At order two the jet is faster by a factor between 1.39 and

1.46 across all seven order-two settings, and it allocates between 1.49 and 1.55 times less memory. At order two every residual term is a pure power, so the schedule uses one direction and the saving is that of Taylor-mode evaluation: nested differentiation retains a graph across two coordinate derivatives. The saving then grows with the order, exactly where the rank reduction begins to apply: at order four the step time falls by a factor of 2.25 and the peak memory by 3.07, and at order six by 4.09 and 6.46. In absolute terms, one step of the sixth-order problem costs 854 milliseconds and 5.6 gibibytes with the jet, against 3.5 seconds and 36.1 gibibytes with nested differentiation.

Two mechanisms compound in that growth, and the design of the study separates them. The jet replaces a graph of depth proportional to the derivative order and the network depth by $p + 1$ coefficient tensors per activation, which is what the memory column measures; this part is available at every order and is what the order-two settings show. The schedule then fixes how many times that jet must be run, at the minimum of Table 3.1: one direction for each pure power at any order, three for the mixed term at order four, five for each mixed term at order six. Nested differentiation has no comparable lever, since it must traverse the retained graph once per coordinate derivative of every term, which is why the ratio in panel (c) of Figure 5.1 widens with the order. Both counts are exact. Table 5.1 shows that on this hardware the reduction survives complex arithmetic, kernel launches, and the backward pass.

TABLE 5.2

What the saved time buys under the same protocol: optimizer steps completed inside the common 1000-second budget, as a three-seed mean, and the held-out relative L^2 error reached, as a three-seed mean $\pm$ sample standard deviation. The lower error in each row is set in bold.

setting	steps completed		relative L^2 error	
	jet	nested	jet	nested
Polyharmonic, $2m = 2$	16058	10973	$3.25 \pm 1.15 \times 10^{-4}$	$5.42 \pm 0.88 \times 10^{-4}$
Polyharmonic, $2m = 4$	3975	1767	$6.55 \pm 0.71 \times 10^{-4}$	$1.43 \pm 0.10 \times 10^{-3}$
Polyharmonic, $2m = 6$	1171	287	$3.36 \pm 1.81 \times 10^{-2}$	$9.18 \pm 3.62 \times 10^{-2}$
Radial chirp, $a = 1$	13730	9835	$3.84 \pm 3.68 \times 10^{-4}$	$7.05 \pm 3.55 \times 10^{-4}$
Radial chirp, $a = 2$	13617	9816	$8.93 \pm 10.97 \times 10^{-4}$	$8.36 \pm 4.56 \times 10^{-4}$
Radial chirp, $a = 3$	13629	9796	$1.86 \pm 0.23 \times 10^{-4}$	$3.38 \pm 1.25 \times 10^{-4}$
Maxwell, $a = 2$	13639	9795	$1.24 \pm 0.44 \times 10^{-3}$	$1.61 \pm 0.72 \times 10^{-3}$
Maxwell, $a = 4$	13665	9814	$1.24 \pm 0.40 \times 10^{-3}$	$1.48 \pm 0.67 \times 10^{-3}$
Maxwell, $a = 6$	13712	9825	$9.27 \pm 1.52 \times 10^{-3}$	$9.42 \pm 1.49 \times 10^{-3}$

5.5. What the saving buys. A cheaper evaluation is useful if more evaluations fit in the same wall-clock budget. Table 5.2 therefore records how many optimizer steps each line completes inside the common budget and the accuracy it reaches, and Figure 5.2 shows the trajectories. The step counts follow the cost directly: at order six the jet completes 1171 steps against 287, and at order four 3975 against 1767. The accuracy follows the step counts where the problem is hard enough for the extra steps to matter. At order four the jet reaches 6.55×10^{-4} against 1.43×10^{-3} , and at order six 3.36×10^{-2} against 9.18×10^{-2} , in both cases with the seed spread of Table 5.2 well inside the gap. Across the nine settings the jet has the lower three-seed mean error in eight; the exception is the chirp at $a = 2$, where the two lines are within one seed standard deviation of each other.

At order two the extra steps buy little: the two backends track each other in panels (b) and (c) of Figure 5.2, and in five of the six frequency-swept settings the

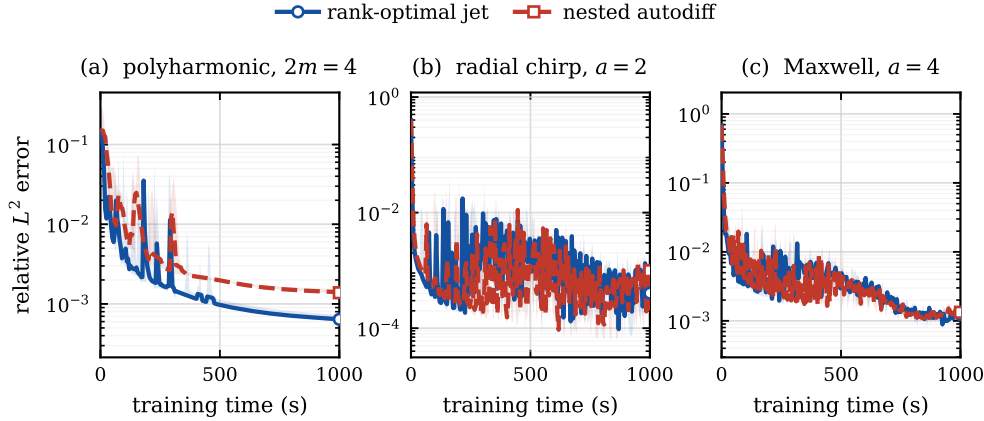


FIG. 5.2. Held-out relative L^2 error against training time for the two backends, on one representative setting of each family. Curves are the three-seed median and the shaded band is the seedwise range. Each panel is framed on the median curves.

two means differ by less than one seed standard deviation. A cheaper derivative is not by itself a better optimizer, and at low order the derivative is not what limits the run. The cost advantage of the schedule grows with the derivative order, in wall time and in memory, with the model, the loss, and the optimizer held fixed.

6. Conclusion. The shortest exact formula for a high-order derivative, in directional Taylor coefficients of the same order, is the Waring decomposition of the corresponding monomial. Carlini, Catalisano, and Geramita evaluate the complex length as (12), and a roots-of-unity formula attains it. No shorter linear combination of order- p directional Taylor coefficients exists; the two-point identity for $\partial_1 \partial_2$ is the length-two case.

Combined with a batched Taylor jet and a closed-form vector-Jacobian product, the schedule evaluates the derivative exactly and remains differentiable in θ . Double-precision tests reproduce nested automatic differentiation for values and for parameter gradients through order six. Measured against nested differentiation, with the function, the loss, and the optimizer held fixed, the schedule reduces the time of one evaluation by factors of about 1.4, 2.3, and 4.1 at derivative orders two, four, and six, and its peak memory by about 1.5, 3.1, and 6.5; on the higher-order problems the evaluations thus gained lower the error reached within a fixed budget. The two mechanisms behind that growth are separable in the measurements: Taylor-mode evaluation removes the nested graph at every order, and the rank reduction shortens the schedule wherever an index repeats.

The schedule is shortest for repeated-index multi-indices and exponentially long for square-free ones: Theorem 3.3 shows that this length is the minimum. At low derivative order a cheaper derivative also buys little in a complete run, since the derivative is not what limits it. Natural continuations are a rank-aware expansion of an operator that reuses directions across its terms, a conditioning analysis of the schedule at high order and in reduced precision, and the extension of the same apolarity argument to derivative tensors contracted against a fixed structured family.

Appendix A. Schedules, filters, and the apolar bound.

This appendix collects the roots-of-unity filter used in the proof of Theorem 3.3, the explicit schedules for the derivative patterns that occur in Section 5, and the apolar interpretation of the rank formula (12). The formulas of Appendix A.2 are those evaluated by the implementation.

A.1. Roots-of-unity filter. LEMMA A.1. *For every integer $m \geq 1$ and every integer k ,*

$$\sum_{\zeta \in U_m} \zeta^k = \begin{cases} m, & m \mid k, \\ 0, & m \nmid k. \end{cases} \quad (23)$$

Proof. Let $\omega := \exp(2\pi i/m)$, so that every element of U_m is ω^ℓ for exactly one $\ell \in \{0, \dots, m-1\}$ and

$$\sum_{\zeta \in U_m} \zeta^k = \sum_{\ell=0}^{m-1} (\omega^k)^\ell.$$

If $m \mid k$, then $\omega^k = 1$ and the sum equals m . If $m \nmid k$, then $\omega^k \neq 1$, and the finite geometric series gives

$$\sum_{\ell=0}^{m-1} (\omega^k)^\ell = \frac{1 - \omega^{km}}{1 - \omega^k} = 0,$$

since $\omega^m = 1$. □

A.2. Explicit low-order schedules. The mixed terms of the polyharmonic residual (20) and the pure powers of the chirp and Maxwell residuals use the following instances of (13).

For a pure power $\partial^\nu = \partial_1^p$ we have $n = 0$, the schedule has one direction, and T_p is evaluated once,

$$\partial_1^p u(\mathbf{x}) = p! T_p(\mathbf{x}; e_1). \quad (24)$$

For the fourth-order mixed term $\partial^\nu = \partial_1^2 \partial_2^2$ we have $n = 1$ and $a = (2, 2)$; with $\omega := \exp(2\pi i/3)$ and $U_3 = \{1, \omega, \omega^2\}$, formula (13) gives the three directions

$$\partial_1^2 \partial_2^2 u(\mathbf{x}) = \frac{4}{3} \sum_{\zeta \in U_3} \zeta T_p(\mathbf{x}; e_1 + \zeta e_2). \quad (25)$$

For the sixth-order mixed term $\partial^\nu = \partial_1^2 \partial_2^4$ we have $n = 1$ and $a = (2, 4)$; with U_5 the fifth roots of unity, (13) takes the coordinate of smaller exponent as base and gives the five directions

$$\partial_1^2 \partial_2^4 u(\mathbf{x}) = \frac{48}{5} \sum_{\zeta \in U_5} \zeta T_p(\mathbf{x}; e_1 + \zeta e_2). \quad (26)$$

The companion term $\partial_1^4 \partial_2^2$ is the same formula with the two coordinates exchanged. The equivalent polynomial identity (10) for (26) reads

$$6! z_0^2 z_1^4 = \frac{48}{5} \sum_{\zeta \in U_5} \zeta (z_0 + \zeta z_1)^6, \quad (27)$$

and it can be checked directly: the coefficient of $z_0^{\beta_0} z_1^{\beta_1}$ on the right-hand side contains the factor $\sum_{\zeta \in U_5} \zeta^{1+\beta_1}$, which by Lemma A.1 is nonzero only when $\beta_1 \equiv 4 \pmod{5}$. Together with $\beta_0 + \beta_1 = 6$ this forces $\beta = (2, 4)$, and the surviving constant is $\frac{48}{5} \cdot \frac{6!}{2!4!} \cdot 5 = 720 = 6!$. The three-variable pattern $(2, 2, 2)$ of Table 3.1 is the same construction with two cube-root factors and nine directions.

A.3. Apolarity interpretation of the rank formula. The rank formula (12) has a standard apolar interpretation. For a homogeneous $F \in \mathbb{C}[\mathbf{z}]_p$, let $F^\perp$ denote the ideal of differential operators that annihilate F . A finite subscheme $X \subset \mathbb{P}^n$ is apolar to F when $I_X \subset F^\perp$. The apolarity lemma characterizes Waring rank as

$$R_{\mathbb{C}}(F) = \min\{\deg X : X \subset \mathbb{P}^n \text{ finite, reduced, and apolar to } F\}. \quad (28)$$

For the monomial $F = \mathbf{z}^a$, the apolar ideal is the complete intersection

$$(\mathbf{z}^a)^\perp = (\partial_0^{a_0+1}, \dots, \partial_n^{a_n+1}), \quad (29)$$

which is an Artinian complete intersection. The step from (29) to the lower bound in (12) uses the Hilbert-function argument in [3]; the general apolarity framework is developed in [15, 19]. The linear forms $\mathbf{v}_\zeta \cdot \mathbf{z}$ defined by (13) determine $\prod_{j=1}^n (a_j + 1)$ distinct projective points. Proposition 3.1 and the apolarity lemma identify these points with a reduced apolar scheme of the degree prescribed by (12).

REFERENCES

- [1] ATILIM GUNES BAYDIN, BARAK A. PEARLMUTTER, ALEXEY ANDREYEVICH RADUL, AND JEFFREY MARK SISKIND, *Automatic differentiation in machine learning: a survey*, Journal of Machine Learning Research, 18 (2018), pp. 1–43.
- [2] JESSE BETTENCOURT, MATTHEW J. JOHNSON, AND DAVID DUVENAUD, *Taylor-mode automatic differentiation for higher-order derivatives in JAX*, in NeurIPS Workshop on Program Transformations for Machine Learning, 2019.
- [3] ENRICO CARLINI, MARIA VIRGINIA CATALISANO, AND ANTHONY V. GERAMITA, *The solution to the Waring problem for monomials and the sum of coprime monomials*, Journal of Algebra, 370 (2012), pp. 5–14.
- [4] PAO-HSIUNG CHIU, JIAN CHENG WONG, CHINCHUN OOI, MY HA DAO, AND YEW-SOON ONG, *CAN-PINN: A fast physics-informed neural network based on coupled-automatic-numerical differentiation method*, Computer Methods in Applied Mechanics and Engineering, 395 (2022), p. 114909.
- [5] PHILIPPE G. CIARLET, *Mathematical Elasticity, Volume II: Theory of Plates*, vol. 27 of Studies in Mathematics and its Applications, North-Holland, Amsterdam, 1997.
- [6] BENGT FORNBERG, *Generation of finite difference formulas on arbitrarily spaced grids*, Mathematics of Computation, 51 (1988), pp. 699–706.
- [7] FILIPPO GAZZOLA, HANS-CHRISTOPH GRUNAU, AND GUIDO SWEERS, *Polyharmonic Boundary Value Problems*, vol. 1991 of Lecture Notes in Mathematics, Springer, Berlin, 2010.
- [8] ANDREAS GRIEWANK, JEAN UTKE, AND ANDREA WALTHER, *Evaluating higher derivative tensors by forward propagation of univariate Taylor series*, Mathematics of Computation, 69 (2000), pp. 1117–1130.
- [9] ANDREAS GRIEWANK AND ANDREA WALTHER, *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation*, Society for Industrial and Applied Mathematics, Philadelphia, PA, 2nd ed., 2008.
- [10] DI HE, SHANDA LI, WENLEI SHI, XIAOTIAN GAO, JIA ZHANG, JIANG BIAN, LIWEI WANG, AND TIE-YAN LIU, *Learning physics-informed neural networks without stacked back-propagation*, in Proceedings of the 26th International Conference on Artificial Intelligence and Statistics, vol. 206 of Proceedings of Machine Learning Research, PMLR, 2023, pp. 3034–3047.
- [11] ZHEYUAN HU, ZEKUN SHI, GEORGE EM KARNIADAKIS, AND KENJI KAWAGUCHI, *Hutchinson trace estimation for high-dimensional and high-order physics-informed neural networks*, Computer Methods in Applied Mechanics and Engineering, 424 (2024), p. 116883.

- [12] ———, *Bias-variance trade-off in physics-informed neural networks with randomized smoothing for high-dimensional PDEs*, Journal of Computational Physics, 521 (2025), p. 113524.
- [13] ZHEYUAN HU, KHEMRAJ SHUKLA, GEORGE EM KARNIADAKIS, AND KENJI KAWAGUCHI, *Tackling the curse of dimensionality with physics-informed neural networks*, Neural Networks, 176 (2024), p. 106369.
- [14] MICHAEL F. HUTCHINSON, *A stochastic estimator of the trace of the influence matrix for Laplacian smoothing splines*, Communications in Statistics: Simulation and Computation, 18 (1989), pp. 1059–1076.
- [15] ANTHONY IARROBINO AND VASSIL KANEV, *Power Sums, Gorenstein Algebras, and Determinantal Loci*, vol. 1721 of Lecture Notes in Mathematics, Springer, Berlin, 1999.
- [16] FRANK IHLENBURG, *Finite Element Analysis of Acoustic Scattering*, vol. 132 of Applied Mathematical Sciences, Springer, New York, 1998.
- [17] GEORGE EM KARNIADAKIS, IOANNIS G. KEVREKIDIS, LU LU, PARIS PERDIKARIS, SIFAN WANG, AND LIU YANG, *Physics-informed machine learning*, Nature Reviews Physics, 3 (2021), pp. 422–440.
- [18] ISAAC E. LAGARIS, ARISTIDIS LIKAS, AND DIMITRIOS I. FOTIADIS, *Artificial neural networks for solving ordinary and partial differential equations*, IEEE Transactions on Neural Networks, 9 (1998), pp. 987–1000.
- [19] JOSEPH M. LANDSBERG, *Tensors: Geometry and Applications*, vol. 128 of Graduate Studies in Mathematics, American Mathematical Society, Providence, RI, 2012.
- [20] RUICHEN LI, HAOTIAN YE, DU JIANG, XUELAN WEN, CHUWEI WANG, ZHE LI, XIANG LI, DI HE, JI CHEN, WEILUO REN, AND LIWEI WANG, *A computational framework for neural network-based variational Monte Carlo with Forward Laplacian*, Nature Machine Intelligence, 6 (2024), pp. 209–219.
- [21] PETER MONK, *Finite Element Methods for Maxwell’s Equations*, Oxford University Press, Oxford, 2003.
- [22] ADAM PASZKE, SAM GROSS, FRANCISCO MASSA, ADAM LERER, JAMES BRADBURY, GREGORY CHANAN, TREVOR KILLEEN, ZEMING LIN, NATALIA GIMELSHEIN, LUCA ANTIGA, ALBAN DESMAISON, ANDREAS KÖPF, EDWARD YANG, ZACHARY DEVITO, MARTIN RAISON, ALYKHAN TEJANI, SASANK CHILAMKURTHY, BENOIT STEINER, LU FANG, JUNJIE BAI, AND SOUMITH CHINTALA, *PyTorch: An imperative style, high-performance deep learning library*, in Advances in Neural Information Processing Systems, vol. 32, 2019.
- [23] BARAK A. PEARLMUTTER, *Fast exact multiplication by the Hessian*, Neural Computation, 6 (1994), pp. 147–160.
- [24] MAZIAR RAISSI, PARIS PERDIKARIS, AND GEORGE E. KARNIADAKIS, *Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations*, Journal of Computational Physics, 378 (2019), pp. 686–707.
- [25] PATRICK J. ROACHE, *Code verification by the method of manufactured solutions*, Journal of Fluids Engineering, 124 (2002), pp. 4–10.
- [26] ZEKUN SHI, ZHEYUAN HU, MIN LIN, AND KENJI KAWAGUCHI, *Stochastic Taylor derivative estimator: Efficient amortization for arbitrary differential operators*, Advances in Neural Information Processing Systems (NeurIPS), 37 (2024), pp. 122316–122353.
- [27] ———, *STDE++: Polynomial-time amortization for linear differential operators*, Journal of Machine Learning Research, 27 (2026), pp. 1–50.
- [28] JUSTIN SIRIGNANO AND KONSTANTINOS SPILIOPOULOS, *DGM: A deep learning algorithm for solving partial differential equations*, Journal of Computational Physics, 375 (2018), pp. 1339–1364.
- [29] CHARLES M. STEIN, *Estimation of the mean of a multivariate normal distribution*, The Annals of Statistics, 9 (1981), pp. 1135–1151.